Universidad Tecnológica de Jalisco

Ingeniería en Desarrollo y Gestión de Software

Diseño Web Integral · Unidad III: Integración de componentes de software para aplicaciones Web



Actividad 3.2 – Codificación Segura

Proyecto integrador: Prisma

EQUIPO

[NOMBRE DEL EQUIPO]

INTEGRANTES

[Nombre completo del integrante 1]

[Nombre completo del integrante 2]

[Nombre completo del integrante 3]

[Nombre completo del integrante 4]

[Nombre completo del integrante 5]

11 de noviembre de 2026

Contenido

Introducción

Parte 1: Protección de Datos Personales y Privacidad

1.1 Minimización y cifrado en almacenamiento — App móvil

1.2 Contraseñas con hash y sal, y datos sensibles cifrados — Backend

1.3 Gestión de cookies y sesiones

1.4 Manejo de logs y depuración

Parte 2: Codificación Segura y Mitigación de Vulnerabilidades (OWASP)

2.1 Validación de entradas (listas blancas) y codificación de salidas (XSS)

2.2 Consultas parametrizadas (prevención de inyección SQL)

2.3 Control de acceso y referencias directas (IDOR)

2.4 Protección contra CSRF

Parte 3: Certificados de Seguridad Web (SSL / TLS)

3.1 HTTPS con TLS 1.2 / 1.3 y redirección forzada

3.2 Cabeceras HTTP de seguridad

3.3 Seguridad en la App Móvil: validación de certificado, Certificate Pinning y manifiesto

Parte 4: Implementación de Web Services

4.1 Arquitectura de servicios: catálogo OpenAPI

4.2 Integración de APIs de terceros

4.3 Autenticación remota segura: JWT

4.4 Codificación segura y auditoría: SAST y DAST

Resumen de cumplimiento

Conclusión

Introducción

Prisma es un asistente de finanzas personales que ayuda a controlar tarjetas de crédito, ingresos y nóminas. La arquitectura tiene tres componentes que comparten el mismo código Kotlin Multiplatform:

- Backend Web API: Kotlin + Ktor 3.6 (Netty) con PostgreSQL mediante el ORM Exposed.
- Cliente Web: Compose Multiplatform compilado a Kotlin/Wasm y servido con webpack.
- App móvil: Android (y iOS) con Compose Multiplatform; cliente HTTP Ktor con OkHttp (Android) y URLSession (iOS).

Este documento muestra, para cada lineamiento de la actividad, qué se implementó en Prisma, el código correspondiente y la evidencia de que funciona: salidas reales de terminal, pruebas automáticas, capturas del emulador y del navegador, y los reportes de Semgrep (SAST) y OWASP ZAP (DAST).

Herramienta	Uso en la actividad
openssl 3 / curl	Verificar versiones TLS aceptadas, redirección HTTP→HTTPS, cabeceras y Certificate Pinning.
Script Python (docs/act3.2/pruebas_seguridad_api.py)	26 ataques reales contra la API: IDOR, CSRF, inyección SQL, XSS, JWT alterado, subida maliciosa.
Pruebas JUnit del servidor (33)	Regresión automática de los controles (./gradlew :server:test).
Semgrep 1.157	Análisis estático de código (SAST) con reglas Kotlin, Java, secretos, OWASP Top 10 y JWT.
OWASP ZAP 2.17	Análisis dinámico (DAST) autenticado con JWT sobre el catálogo OpenAPI: escaneo pasivo y activo.
Emulador Android (Pixel 10a) + adb	Conexión HTTPS con pinning y simulación de ataque Man-in-the-Middle.

Parte 1: Protección de Datos Personales y Privacidad

1.1 Minimización y cifrado en almacenamiento — App móvil

La app no guarda contraseñas ni el token de sesión en el teléfono: el JWT vive solo en memoria mientras la app está abierta. Lo único que se persiste es el correo del usuario y el token del dispositivo para entrar con huella. Antes se guardaban en un SharedPreferences normal; ahora se guardan en EncryptedSharedPreferences:

- Llaves cifradas con AES-256-SIV y valores con AES-256-GCM.
- Llave maestra AES-256 dentro del Android Keystore (no se puede extraer del teléfono).
- Archivo con MODE_PRIVATE (solo la app puede leerlo) y nunca en la tarjeta SD.
- Migración automática: si existía el archivo viejo sin cifrar, se copia y se borra.
- El token del dispositivo, además, va cifrado con AES-256-GCM con una llave del Keystore que exige la huella para usarse.

SecurePrefs.kt — EncryptedSharedPreferences con llave en Android Keystore `app/shared/src/androidMain/kotlin/com/example/prisma/storage/SecurePrefs.kt:22`

```
fun open(context: Context, legacyName: String? = null): SharedPreferences {
    val app = context.applicationContext
    val masterKey = MasterKey.Builder(app)
        .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
        .build()
    val encrypted = EncryptedSharedPreferences.create(
        app,
        ENCRYPTED_FILE,
        masterKey,
        EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
        EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM,
    )
    if (legacyName != null) migrate(app.getSharedPreferences(legacyName, Context.MODE_PRIVATE), encryp
ted, app, legacyName)
    return encrypted
```

BiometricAuth.android.kt — uso del almacenamiento cifrado `app/shared/src/androidMain/kotlin/com/example/prisma/biometric/BiometricAuth.android.kt:50`

```
// EncryptedSharedPreferences (AES-256, llave en Android Keystore, MODE_PRIVATE). Migra el archivo vie
jo sin cifrar.
private val prefs = SecurePrefs.open(activity, legacyName = "prisma_biometric")
```

También se deshabilitaron los respaldos automáticos (allowBackup=false y dataExtractionRules), para que los datos de la app no salgan del teléfono hacia la nube ni a otro dispositivo (ver sección 3.3).

1.2 Contraseñas con hash y sal, y datos sensibles cifrados — Backend

Contraseñas: BCrypt (costo 12, sal aleatoria por usuario)

Passwords.kt — hash BCrypt `server/src/main/kotlin/com/example/prisma/auth/Passwords.kt:12`

```
fun hash(password: String): String = BCrypt.withDefaults().hashToString(COST, password.toCharArray())
```

BCrypt genera una sal aleatoria de 128 bits para cada contraseña y la guarda dentro del propio hash. Por eso dos usuarios con la misma contraseña tienen hashes distintos, como se ve en la evidencia de la base de datos.

Datos personales en reposo: AES-256-GCM

El nombre y el apellido del perfil se cifran antes de guardarse en PostgreSQL. La llave de 256 bits está en DATA_ENCRYPTION_KEY (archivo server/.env, fuera del repositorio). Cada valor usa un IV aleatorio, y la etiqueta GCM detecta cualquier alteración del dato guardado.

DataCipher.kt — AES-256-GCM `server/src/main/kotlin/com/example/prisma/security/DataCipher.kt:42`

```
class AesGcmCipher(key: ByteArray) {
    init {
        require(key.size == 32) { "La llave AES-256 debe medir 32 bytes" }
    }

    private val secretKey = SecretKeySpec(key, "AES")
    private val random = SecureRandom()

    fun encrypt(plain: String): String {
        val iv = ByteArray(IV_BYTES).also(random::nextBytes)
        val cipher = Cipher.getInstance(TRANSFORMATION).apply { init(Cipher.ENCRYPT_MODE, secretKey, G
CMParameterSpec(TAG_BITS, iv)) }
        val sealed = cipher.doFinal(plain.toByteArray(Charsets.UTF_8))
        return PREFIX + Base64.getEncoder().encodeToString(iv + sealed)
    }

    fun decrypt(stored: String): String {
        if (!stored.startsWith(PREFIX)) return stored // dato heredado sin cifrar
        val bytes = Base64.getDecoder().decode(stored.removePrefix(PREFIX))
        val cipher = Cipher.getInstance(TRANSFORMATION).apply {
            init(Cipher.DECRYPT_MODE, secretKey, GCMParameterSpec(TAG_BITS, bytes, 0, IV_BYTES))
        }
        return cipher.doFinal(bytes, IV_BYTES, bytes.size - IV_BYTES).toString(Charsets.UTF_8)
    }
```

ProfileRoutes.kt — cifrado al guardar `server/src/main/kotlin/com/example/prisma/routes/ProfileRoutes.kt:94`

```
// PII cifrada en reposo con AES-256-GCM.
it[DatosPersonalesTable.nombre] = DataCipher.encrypt(nombre)
it[DatosPersonalesTable.apellido] = DataCipher.encrypt(apellido)
```

EVIDENCIA **PostgreSQL: nombre/apellido cifrados, contraseñas con BCrypt + sal (06-bd-cifrado-y-hash.txt)**

```
$ psql prisma_db  (usuario de prueba: alice.82c227@prisma.test)

-- Datos personales tal como quedan guardados en PostgreSQL (AES-256-GCM):
                          nombre                          |                          apellido
----------------------------------------------------------+-----------------------------------------------
```

```
--------------
 enc:v1:tvbGTEELRc+5wJ8VQZW6JTn6J6JunNS/lzm8UOPyKCHaiA== | enc:v1:El38a252aMSVmU92psfotYeozybg4BXU8zz
1mHNq+hfXYw==
(1 row)

-- La API se los devuelve descifrados solo a su dueño: {"nombre":"Alicia","apellido":"Prueba",...}

-- Contraseñas: hash BCrypt ($2a$ = algoritmo, 12 = costo 2^12, los siguientes 22 caracteres = sal al
eatoria):
    correo_electronico    |                      hash_contrasena
--------------------------+------------------------------------------------------------
 alice.82c227@prisma.test | $2a$12$9nTDq5kVu2OGhpCmuUW08Ocdtyde3S3NA3JcwBDeTVcSX3Xr90fXa
 bob.82c227@prisma.test   | $2a$12$cR3RnwyQB1zOhTgHivI9g.FSW4rr6kHN.g8W5Xp1X7kX0fZaoBWJq
(2 rows)

-- Misma contraseña 'Prueba#Segura2026' en ambos usuarios → hashes distintos gracias a la sal.

-- Consentimientos registrados al crear la cuenta:
  consentimiento  | aceptado | obligatorio | version_documento
------------------+----------+-------------+-------------------
 AvisoPrivacidad  | t        | t           | 2026-11
 DatosFinancieros | t        | t           | 2026-11
 EnvioIa          | t        | t           | 2026-11
 TerminosUso      | t        | t           | 2026-11
 Encuestas        | f        | f           | 2026-11
 Estadisticas     | f        | f           | 2026-11
 Novedades        | f        | f           | 2026-11
(7 rows)
```

1.3 Gestión de cookies y sesiones

Ktor no usa express-session; el equivalente se implementó en CallHelpers.kt. Al verificar el código 2FA, el servidor entrega el JWT en el cuerpo (para las apps móviles) y, para el navegador, en una cookie con estos atributos:

- HttpOnly: JavaScript no puede leerla, así que un XSS no puede robar la sesión.
- Secure: solo viaja por HTTPS.
- SameSite=Strict: el navegador no la envía en peticiones que inicia otro sitio.
- Prefijo __Host-: obliga a Secure y Path=/ y prohíbe el atributo Domain.

CallHelpers.kt — cabecera Set-Cookie `server/src/main/kotlin/com/example/prisma/http/CallHelpers.kt:97`

```
private fun ApplicationCall.setCookie(name: String, value: String, httpOnly: Boolean, maxAge: Int) {
    val attributes = buildList {
        add("$name=$value")
        add("Max-Age=$maxAge")
        add("Path=/")
        add("Secure")
        if (httpOnly) add("HttpOnly")
        add("SameSite=Strict")
    }
    response.headers.append(HttpHeaders.SetCookie, attributes.joinToString("; "))
```

EVIDENCIA cabeceras Set-Cookie reales del servidor (05-pruebas-ataques-api.txt)

```
=== 2. Cookies de sesión (Parte 1.2) ===
Set-Cookie: __Host-prisma_session=<valor>; Max-Age=43200; Path=/; Secure; HttpOnly; SameSite=Strict
```

```
Set-Cookie: __Host-prisma_csrf=<valor>; Max-Age=43200; Path=/; Secure; SameSite=Strict
[OK ] Cookie de sesión con HttpOnly, Secure y SameSite=Strict
        flags presentes
```

CAPTURA PENDIENTE — COOKIES EN EL NAVEGADOR

1. Abre una vez https://localhost:8443 en Chrome y acepta el certificado (Avanzado → Continuar).
2. Abre http://localhost:3001, inicia sesión con tu cuenta y el código 2FA.
3. DevTools (F12) → Application → Cookies → https://localhost:8443: toma la captura donde se vean las columnas HttpOnly ✓, Secure ✓ y SameSite = Strict de __Host-prisma_session.

Inserta aquí la captura.

1.4 Manejo de logs y depuración

Ktor no tiene una bandera debug como Django; su equivalente es developmentMode, que queda en false. Además se agregó PRISMA_ENV (production por defecto):

- StatusPages: cualquier excepción responde un JSON genérico; la traza solo queda en el log interno del servidor.
- logback.xml: el nivel pasó de TRACE a INFO. Ya no se registran las sentencias SQL con sus valores (logger Exposed).
- Los correos se enmascaran en el log (p***@gmail.com). En producción nunca se imprimen códigos 2FA ni el cuerpo de los correos.
- Nunca se registran contraseñas, tokens ni cuerpos de peticiones.

Application.kt — errores sin trazas (debug = false) `server/src/main/kotlin/com/example/prisma/Application.kt:133`

```
// ---------- Errores sin trazas (debug = false) ----------
// El cliente solo recibe un mensaje genérico; la traza completa queda únicamente en el log del servid
or.
install(StatusPages) {
    exception<Throwable> { call, cause ->
        call.application.log.error("Error no controlado en ${call.request.path()}: ${cause::class.simp
leName}", cause)
        call.respond(HttpStatusCode.InternalServerError, ApiError("internal_error", "Ocurrió un error
interno. Inténtalo más tarde."))
    }
}
```

logback.xml — nivel de producción `server/src/main/resources/logback.xml:12`

```
<root level="INFO">
    <appender-ref ref="STDOUT"/>
</root>
<logger name="Exposed" level="WARN"/>
<logger name="com.zaxxer.hikari" level="WARN"/>
```

```
<logger name="org.eclipse.jetty" level="INFO"/>
<logger name="io.netty" level="WARN"/>
```

EVIDENCIA log de producción tras logins fallidos con contraseña: sin datos sensibles (07-logs-produccion.txt)

```
$ cat server.log   (servidor en PRISMA_ENV=production tras recibir logins fallidos con contraseña)
2026-11-11 17:34:26.023 [main] INFO  io.ktor.server.Application - Autoreload is disabled because the
development mode is off.
2026-11-11 17:34:26.362 [main] INFO  io.ktor.server.Application - Correo SMTP configurado: p***@gmai
l.com vía smtp.gmail.com.
2026-11-11 17:34:26.364 [main] INFO  io.ktor.server.Application - Entorno: production · modo depuraci
ón de Ktor: false
2026-11-11 17:34:26.508 [main] INFO  io.ktor.server.Application - Application started in 0.627 second
s.
2026-11-11 17:34:26.954 [DefaultDispatcher-worker-1] INFO  io.ktor.server.Application - Responding at
http://0.0.0.0:8080
2026-11-11 17:34:26.954 [DefaultDispatcher-worker-1] INFO  io.ktor.server.Application - Responding at
https://0.0.0.0:8443

$ grep -ciE 'MiPasswordSecreto|nadie@prisma|Bearer|eyJ|password=' server.log
0
```

EVIDENCIA prueba automática: un error interno no revela la traza (SecurityTest.kt)

```
fun internalErrorsDoNotLeakStackTraces() = withApp {
    val response = client.get("/prueba-error")
    assertEquals(HttpStatusCode.InternalServerError, response.status)
    val body = response.bodyAsText()
    assertContains(body, "internal_error")
    assertFalse("password" in body || "com.example" in body || "Exception" in body || "at " in body,
body)
}
```

Parte 2: Codificación Segura y Mitigación de Vulnerabilidades (OWASP)

2.1 Validación de entradas (listas blancas) y codificación de salidas (XSS)

Cada endpoint valida con listas blancas el formato, el tipo, el rango y los caracteres permitidos antes de tocar la base de datos. Los cuerpos JSON se mapean a DTOs tipados de kotlinx.serialization; un JSON con tipos incorrectos se rechaza con 400.

Campo	Regla de lista blanca
Correo	Formato usuario@dominio, máx. 255 caracteres, en minúsculas
Contraseña	Mínimo 8 caracteres
Nombre / apellido	^[\p{L}][\p{L} .'\-]*$ — solo letras, espacio, apóstrofo y guion; máx. 100
Banco, alias, nombre de ingreso	^[\p{L}\p{N}][\p{L}\p{N} .,()/#&+\-]*$ — sin < > " ` { } ;
Últimos 4 dígitos	Exactamente 4 números
Monto	Número > 0, máx. 2 decimales, hasta 9,999,999,999.99
Ids en la URL	Deben ser UUID válidos
Código 2FA	6 dígitos
Plataforma biométrica	android | ios
Archivos subidos	PDF/JPG/PNG/WEBP/HEIC por su contenido real (magic bytes), máx. 10 MB, nombre saneado

TextValidation.kt — listas blancas de texto `server/src/main/kotlin/com/example/prisma/http/TextValidation.kt:10`

```
object TextValidation {
    /** Nombres de personas: letras (con acentos y ñ), espacio, apóstrofo, guion y punto. */
    private val PERSON_NAME = Regex("^[\\p{L}][\\p{L} .'\\-]*$")

    /** Etiquetas (banco, alias de tarjeta, nombre de ingreso): letras, números y puntuación básica.
*/
    private val LABEL = Regex("^[\\p{L}\\p{N}][\\p{L}\\p{N} .,()/#&+\\-]*$")

    fun isPersonName(value: String): Boolean = PERSON_NAME.matches(value)

    fun isLabel(value: String): Boolean = LABEL.matches(value)
```

UploadValidation.kt — el tipo se decide por el contenido, no por lo que dice el cliente `server/src/main/kotlin/com/example/prisma/http/UploadValidation.kt:28`

```
fun detectMime(bytes: ByteArray): String? {
    fun startsWith(vararg sig: Int) = bytes.size >= sig.size && sig.indices.all { bytes[it] == sig[i
t].toByte() }
    fun asciiAt(offset: Int, text: String) =
        bytes.size >= offset + text.length && text.indices.all { bytes[offset + it] == text[it].code.t
oByte() }
```

```
    return when {
        asciiAt(0, "%PDF-") -> "application/pdf"
        startsWith(0xFF, 0xD8, 0xFF) -> "image/jpeg"
        startsWith(0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A, 0x1A, 0x0A) -> "image/png"
        asciiAt(0, "RIFF") && asciiAt(8, "WEBP") -> "image/webp"
        asciiAt(4, "ftyp") && (asciiAt(8, "heic") || asciiAt(8, "heix") || asciiAt(8, "mif1")) -> "ima
ge/heic"
        else -> null
    }
}
```

Codificación de salida: la API solo responde JSON (Content-Type: application/json) serializado con kotlinx.serialization, que escapa las comillas y los caracteres de control. Además envía X-Content-Type-Options: nosniff y Content-Security-Policy: default-src 'none', así que el navegador nunca interpreta una respuesta como HTML o script. En los clientes, Compose dibuja el texto en un canvas y nunca lo inserta como HTML, por lo que no existe un innerHTML que explotar. En la web, la CSP prohíbe además scripts externos y eval().

EVIDENCIA intentos de XSS y archivos maliciosos rechazados (05-pruebas-ataques-api.txt)

```
=== 6. XSS — validación de entradas y salida JSON (Parte 2.1) ===
[OK ] perfil nombre='<script>alert(document.cookie)</script>' → 400
        400 {"code":"invalid_request","message":"Nombre y apellido son obligatorios: solo letras, esp
acios, apóstrofo o gu…
[OK ] perfil apellido='"><img src=x onerror=alert(1)>' → 400
        400 {"code":"invalid_request","message":"Nombre y apellido son obligatorios: solo letras, esp
acios, apóstrofo o gu…
[OK ] banco con <svg onload> → 400
        400 {"code":"invalid_request","message":"Nombre de banco inválido."}
[OK ] Las respuestas son JSON con nosniff y CSP default-src 'none'
        application/json

=== 7. Subida de archivos con lista blanca (Parte 2.1) ===
[OK ] HTML disfrazado de PDF (estado.pdf) → 415
        415 {"code":"invalid_request","message":"Solo se aceptan archivos PDF, JPG, PNG, WEBP o HEI
C."}
[OK ] PDF real con nombre malicioso → se guarda con nombre limpio
        201 nombreArchivo='b_estado.pdf'
```

2.2 Consultas parametrizadas (prevención de inyección SQL)

Todo el acceso a PostgreSQL usa el DSL del ORM Exposed, que genera sentencias preparadas (PreparedStatement) con parámetros. No hay concatenación de SQL. Semgrep lo confirma: 0 hallazgos de inyección.

AuthRoutes.kt — búsqueda del usuario (parámetro enlazado, no concatenado) `server/src/main/kotlin/com/example/prisma/routes/AuthRoutes.kt:131`

```
val user = dbQuery {
    UsuariosTable.selectAll()
        .where { UsuariosTable.correoElectronico.lowerCase() eq email }
        .singleOrNull()
        ?.let { it[UsuariosTable.id] to it[UsuariosTable.hashContrasena] }
}
```

Las migraciones SQL (carpeta db/migrations) son scripts fijos sin datos del usuario. El parser de nóminas XML desactiva DTD y entidades externas (protección XXE).

EVIDENCIA **pruebas de inyección fallidas contra la API real (05-pruebas-ataques-api.txt)**

```
=== 5. Inyección SQL (Parte 2.2) ===
[OK ] login con email="' OR '1'='1' --"
        401 {"code":"invalid_credentials","message":"Correo o contraseña incorrectos."}
[OK ] login con email="admin@prisma.test' --"
        401 {"code":"invalid_credentials","message":"Correo o contraseña incorrectos."}
[OK ] login con email="x@x.com'; DROP TABLE usuarios; --"
        401 {"code":"invalid_credentials","message":"Correo o contraseña incorrectos."}
[OK ] id de ruta '1 OR 1=1' → 400 (no es UUID)
        400 {"code":"invalid_request","message":"Id inválido."}
[OK ] nombre de ingreso con SQL → 400 (lista blanca)
        400 {"code":"invalid_request","message":"El nombre del ingreso es obligatorio: letras, número
s y . , ( ) / # & + -…
[OK ] monto '1 OR 1=1' → 400 (solo números con 2 decimales)
        400 {"code":"invalid_request","message":"El monto debe ser mayor a 0, con máximo 2 decimales
y hasta 9999999999.99…
[OK ] Los ingresos de Alice siguen ahí (no se ejecutó DELETE)
        {"ingresos":[{"id":"8dfdf6c9-d199-4135-81ed-96dd4df7d4cf","nombre":"Sueldo","montoNeto":"1500
0.00","esAuditado…
```

2.3 Control de acceso y referencias directas (IDOR)

Ningún endpoint confía en el id de la URL. El usuario se obtiene del JWT verificado, y cada consulta filtra por id del recurso Y por usuario dueño. Si el recurso existe pero es de otra persona, la respuesta es la misma que si no existiera (404), así no se revela que existe.

IncomeRoutes.kt — editar solo si el ingreso es del usuario autenticado `server/src/main/kotlin/com/example/prisma/routes/IncomeRoutes.kt:137`

```
// El filtro por usuario impide editar ingresos ajenos aunque se conozca su id.
val updated = dbQuery {
    val owned = (IngresosTable.id eq incomeId) and (IngresosTable.usuarioId eq userId)
```

EVIDENCIA **Bob intenta leer, editar y borrar la tarjeta de Alice (05-pruebas-ataques-api.txt)**

```
=== 4. IDOR — control de acceso por recurso (Parte 2.3) ===
Alice crea la tarjeta 7a7e92ff-28bb-41cb-bff0-689c8a4b5372
[OK ] Bob (GET /api/v1/cards/{id-de-Alice}/documents) → 404
        404 {"code":"not_found","message":"Tarjeta no encontrada."}
[OK ] Bob (PUT /api/v1/cards/{id-de-Alice}) → 404
        404 {"code":"not_found","message":"Tarjeta no encontrada."}
[OK ] Bob (DELETE /api/v1/cards/{id-de-Alice}) → 404
        404 {"code":"not_found","message":"Tarjeta no encontrada."}
[OK ] La tarjeta de Alice sigue intacta
        [{"id":"7a7e92ff-28bb-41cb-bff0-689c8a4b5372","banco":"BBVA","alias":"Tarjeta de Alice","ulti
mosCuatro":"4321"…
[OK ] Bob solo ve sus propias tarjetas (lista vacía)
        200 []
```

2.4 Protección contra CSRF

Cada sesión tiene un token anti-CSRF de 256 bits, aleatorio e impredecible, generado con SecureRandom. Toda petición POST, PUT, PATCH o DELETE autenticada con la cookie de sesión debe incluir la cabecera X-CSRF-Token con ese valor; si falta o no coincide, el servidor responde 403. La comparación es en tiempo constante. Las apps móviles envían el JWT en la cabecera Authorization, que el navegador nunca agrega por su cuenta, así que esa vía no es vulnerable a CSRF.

CallHelpers.kt — verificación del token anti-CSRF `server/src/main/kotlin/com/example/prisma/http/CallHelpers.kt:78`

```
if (fromCookie && request.httpMethod in WRITE_METHODS) {
    val sent = request.headers[CSRF_HEADER].orEmpty()
    if (sent.isEmpty() || !constantTimeEquals(sent, info.csrfToken)) {
        respondError(HttpStatusCode.Forbidden, ApiErrorCodes.CSRF_INVALID, "Token anti-CSRF ausente o
inválido.")
        return null
    }
}
```

EVIDENCIA **CSRF contra la API real (05-pruebas-ataques-api.txt)**

```
=== 3. CSRF (Parte 2.4) ===
[OK ] POST con la cookie pero SIN X-CSRF-Token (sitio atacante) → 403
        403 {"code":"csrf_invalid","message":"Token anti-CSRF ausente o inválido."}
[OK ] POST con X-CSRF-Token adivinado → 403
        403 {"code":"csrf_invalid","message":"Token anti-CSRF ausente o inválido."}
[OK ] POST legítimo con la cookie + token CSRF de la sesión → 201
        201 {"id":"8dfdf6c9-d199-4135-81ed-96dd4df7d4cf","nombre":"Sueldo","montoNeto":"15000.00","es
Auditado":false,"fech…
```

Parte 3: Certificados de Seguridad Web (SSL / TLS)

3.1 HTTPS con TLS 1.2 / 1.3 y redirección forzada

Se generó un certificado autofirmado (para pruebas internas) con llave EC P-256, SHA-256 y nombres alternativos para localhost, 127.0.0.1, 10.0.2.2 (emulador) y la IP de red del servidor. Lo crea el script server/certs/generate-cert.sh, y la llave privada (prisma-keystore.p12) está excluida de git.

El servidor abre dos conectores. El 8443 (HTTPS) acepta solo TLSv1.3 y TLSv1.2, así que SSLv2, SSLv3, TLS 1.0 y TLS 1.1 quedan deshabilitados. El 8080 (HTTP) solo existe para responder 301 hacia HTTPS.

Application.kt — conector TLS restringido `server/src/main/kotlin/com/example/prisma/Application.kt:49`

```
/** Solo TLS 1.3 y TLS 1.2. SSLv2, SSLv3, TLS 1.0 y TLS 1.1 quedan deshabilitados. */
private val TLS_PROTOCOLS = listOf("TLSv1.3", "TLSv1.2")
```

Application.kt — conectores HTTP y HTTPS `server/src/main/kotlin/com/example/prisma/Application.kt:64`

```
    // HTTP solo existe para redirigir a HTTPS (HttpsRedirect responde 301 a todo).
    connector {
        host = "0.0.0.0"
        port = HTTP_PORT
    }
    // 0.0.0.0: acepta conexiones de cualquier dispositivo de la red, no solo de esta computadora.
    sslConnector(
        keyStore = keyStore,
        keyAlias = "prisma",
        keyStorePassword = { keystorePassword.toCharArray() },
        privateKeyPassword = { keystorePassword.toCharArray() },
    ) {
        host = "0.0.0.0"
        port = HTTPS_PORT
        keyStorePath = keystoreFile
        enabledProtocols = TLS_PROTOCOLS
    }
},
module = { module() },
```

Application.kt — redirección HTTP → HTTPS `server/src/main/kotlin/com/example/prisma/Application.kt:121`

```
// ---------- HTTPS obligatorio ----------
if (enforceHttps) {
    // Toda petición por HTTP recibe 301 → https://…:8443 con la misma ruta.
    install(HttpsRedirect) {
        sslPort = HTTPS_PORT
        permanentRedirect = true
    }
}
```

EVIDENCIA versiones de TLS aceptadas y rechazadas (01-tls-versiones.txt)

```
$ openssl s_client -connect localhost:8443 -tls1_3
New, TLSv1.3, Cipher is TLS_AES_128_GCM_SHA256
Protocol: TLSv1.3
Verify return code: 18 (self-signed certificate)
```

```
$ openssl s_client -connect localhost:8443 -tls1_2
New, TLSv1.2, Cipher is ECDHE-ECDSA-AES128-GCM-SHA256
Protocol: TLSv1.2
    Protocol  : TLSv1.2

$ openssl s_client -connect localhost:8443 -tls1_1 -cipher DEFAULT@SECLEVEL=0   (TLS 1.1: debe falla
r)
802112F201000000:error:0A00042E:SSL routines:ssl3_read_bytes:tlsv1 alert protocol version:ssl/record/
rec_layer_s3.c:923:SSL alert number 70
SSL handshake has read 7 bytes and written 109 bytes
New, (NONE), Cipher is (NONE)

$ openssl s_client -connect localhost:8443 -tls1 -cipher DEFAULT@SECLEVEL=0   (TLS 1.0: debe fallar)
802112F201000000:error:0A00042E:SSL routines:ssl3_read_bytes:tlsv1 alert protocol version:ssl/record/
rec_layer_s3.c:923:SSL alert number 70
SSL handshake has read 7 bytes and written 106 bytes
New, (NONE), Cipher is (NONE)

$ openssl s_client -connect localhost:8443 -ssl3   (SSLv3: debe fallar)
s_client: Unknown option: -ssl3
```

Nota: openssl 3 ya no incluye soporte de cliente para SSLv3, por eso no se puede ni intentar. Del lado del servidor, SSLv3 está fuera de enabledProtocols y además viene deshabilitado en la JVM.

EVIDENCIA redirección forzada (02-redireccion-https.txt)

```
$ curl -sI http://localhost:8080/api/v1/cards     (HTTP → HTTPS)
HTTP/1.1 301 Moved Permanently
Location: https://localhost:8443/api/v1/cards
```

EVIDENCIA certificado del servidor (04-certificado-y-pinning.txt)

```
$ openssl x509 -in server/certs/prisma-cert.pem -noout -subject -issuer -dates -ext subjectAltName
subject=C=MX, ST=Jalisco, O=Prisma, CN=prisma-api
issuer=C=MX, ST=Jalisco, O=Prisma, CN=prisma-api
notBefore=Nov 11 23:25:16 2026 GMT
notAfter=Feb 13 23:25:16 2029 GMT
X509v3 Subject Alternative Name:
    DNS:localhost, IP Address:127.0.0.1, IP Address:10.0.2.2, IP Address:192.168.100.31
```

3.2 Cabeceras HTTP de seguridad

Se implementó un plugin propio, SecurityHeaders, equivalente a Helmet en Node.js. Se aplica a todas las respuestas de la API, incluidas las de error, las de CORS y las redirecciones. El servidor web de la app (webpack) también envía CSP, X-Frame-Options y nosniff.

Cabecera	Valor	Protege contra
Strict-Transport-Security	max-age=31536000; includeSubDomains	Degradación a HTTP (1 año)
Content-Security-Policy (API)	default-src 'none'; frame-ancestors 'none'	Carga de scripts/imágenes y XSS
Content-Security-Policy (web)	default-src 'self'; script-src 'self' 'wasm-unsafe-eval'; connect-src 'self' https:	Scripts externos, eval(), conexiones HTTP
X-Frame-Options	DENY	Clickjacking
X-Content-Type-Options	nosniff	Interpretación errónea de MIME

Cabecera	Valor	Protege contra
Referrer-Policy / Permissions-Policy	no-referrer / camera=(), …	Fugas de URL y uso de sensores
Server	Prisma	No revela framework ni versión

SecurityHeaders.kt `server/src/main/kotlin/com/example/prisma/security/SecurityHeaders.kt:18`

```
// onCallRespond: se ejecuta justo antes de enviar CUALQUIER respuesta (rutas, errores, CORS,
// redirecciones), incluso si otro plugin la generó antes de llegar a las rutas.
onCallRespond { call, _ ->
    val headers = call.response.headers
    if (headers.contains("Content-Security-Policy")) return@onCallRespond
    val csp = if (call.request.path().startsWith("/swagger")) SWAGGER_CSP else API_CSP
    headers.append("Content-Security-Policy", csp)
    headers.append("X-Frame-Options", "DENY")
    headers.append("X-Content-Type-Options", "nosniff")
    headers.append("Referrer-Policy", "no-referrer")
    headers.append("Permissions-Policy", "camera=(), microphone=(), geolocation=(), payment=()")
    headers.append("Cross-Origin-Opener-Policy", "same-origin")
    headers.append("Cross-Origin-Resource-Policy", "same-site")
    headers.append(HttpHeaders.CacheControl, "no-store")
    headers.append(HttpHeaders.Server, "Prisma", safeOnly = false)
    if (call.request.origin.scheme == "https") {
        headers.append(HttpHeaders.StrictTransportSecurity, "max-age=31536000; includeSubDomains")
    }
}
```

EVIDENCIA **cabeceras reales de la API (03-cabeceras-seguridad.txt)**

```
$ curl -sk -D - -o /dev/null https://localhost:8443/api/v1/cards
HTTP/2 401
vary: Origin
content-security-policy: default-src 'none'; frame-ancestors 'none'; base-uri 'none'; form-action 'no
ne'
x-frame-options: DENY
x-content-type-options: nosniff
referrer-policy: no-referrer
permissions-policy: camera=(), microphone=(), geolocation=(), payment=()
cross-origin-opener-policy: same-origin
cross-origin-resource-policy: same-site
cache-control: no-store
server: Prisma
strict-transport-security: max-age=31536000; includeSubDomains
content-length: 65
content-type: application/json

$ curl -s -D - -o /dev/null http://localhost:8080/api/v1/cards   (HTTP: solo redirección 301, sin HST
S)
HTTP/1.1 301 Moved Permanently
Location: https://localhost:8443/api/v1/cards
```



Cliente web (Kotlin/Wasm) cargado con CSP estricta, autenticándose contra la API por HTTPS

EVIDENCIA inspección de red del navegador (Chrome DevTools Protocol): HTTP/2 + TLS 1.3 y cabeceras (10-web-red-https.txt)

```
red: 200 https://localhost:8443/api/v1/auth/login · h2 · TLS 1.3 AES_128_GCM · cert=prisma-api · seguridad=insecure
    content-security-policy: default-src 'none'; frame-ancestors 'none'; base-uri 'none'; form-action 'none'
    strict-transport-security: max-age=31536000; includeSubDomains
    x-content-type-options: nosniff
    x-frame-options: DENY
red: 401 https://localhost:8443/api/v1/auth/login · h2 · TLS 1.3 AES_128_GCM · cert=prisma-api · seguridad=insecure
    strict-transport-security: max-age=31536000; includeSubDomains
    content-security-policy: default-src 'none'; frame-ancestors 'none'; base-uri 'none'; form-action 'none'
    x-content-type-options: nosniff
    x-frame-options: DENY
```

CAPTURA PENDIENTE — PESTAÑA SECURITY DEL NAVEGADOR

1. En Chrome abre https://localhost:8443/api/v1/cards y acepta el certificado autofirmado.
2. DevTools (F12) → Security (o Privacy and security) → Overview: captura donde diga Connection - secure (TLS 1.3).
3. DevTools → Network → recarga → selecciona la petición → Headers: captura de Strict-Transport-Security, Content-Security-Policy, X-Frame-Options y X-Content-Type-Options.
4. Opcional: para que Chrome muestre el candado sin aviso, confía en server/certs/prisma-cert.pem (Acceso a Llaveros → Sistema → Confiar siempre).

Inserta aquí la captura.

3.3 Seguridad en la App Móvil: validación de certificado, Certificate Pinning y manifiesto

La app de Android usa OkHttp con tres capas de protección:

- TrustManager propio que solo confía en el certificado de Prisma: se rechaza cualquier otro, aunque lo firme una CA instalada en el teléfono.
- Verificación de nombre (hostname): la IP del servidor debe venir en el certificado.
- Certificate Pinning: CertificatePinner compara el SHA-256 de la llave pública del servidor con el pin calculado al compilar.
- Solo TLS 1.3 / 1.2. El tráfico HTTP sin cifrar está prohibido en el manifiesto.

En iOS (URLSession), el certificado presentado debe ser idéntico, byte a byte, al de Prisma; si no, se cancela la conexión.

PlatformHttpClient.android.kt — OkHttp con pinning `app/shared/src/androidMain/kotlin/com/example/prisma/network/PlatformHttpClient.android.kt:29`

```
internal actual fun createPlatformHttpClient(config: HttpClientConfig<*>.() -> Unit): HttpClient =
    HttpClient(OkHttp) {
        config()
        engine {
            config {
                val trustManager = prismaTrustManager()
                val sslContext = SSLContext.getInstance("TLS").apply { init(null, arrayOf(trustManage
r), null) }
                sslSocketFactory(sslContext.socketFactory, trustManager)
                certificatePinner(
                    CertificatePinner.Builder()
                        .add(SERVER_HOST, SERVER_CERT_PIN)
                        .add("10.0.2.2", SERVER_CERT_PIN) // emulador de Android → computadora
                        .add("localhost", SERVER_CERT_PIN)
                        .build(),
                )
                connectionSpecs(
                    listOf(
                        ConnectionSpec.Builder(ConnectionSpec.MODERN_TLS)
```

```
                            .tlsVersions(TlsVersion.TLS_1_3, TlsVersion.TLS_1_2)
                            .build(),
                    ),
                )
            }
        }
```

El pin no se escribe a mano: Gradle lo calcula del certificado al compilar (generateServerConfig) y lo deja en ServerConfig.kt. El valor coincide con el que calcula openssl:

EVIDENCIA pin verificado con curl: el pin correcto conecta y uno distinto se rechaza (04-certificado-y-pinning.txt)

```
Pin esperado (SHA-256 de la llave pública del certificado): sha256//QkBEhKup+2M+O7FXU2UwkK3L+qwHseHlsDF+4F4uuUE=

$ curl --cacert server/certs/prisma-cert.pem --pinnedpubkey "sha256//<pin correcto>" https://localhost:8443/
Prisma API   (exit 0)

$ curl --cacert server/certs/prisma-cert.pem --pinnedpubkey "sha256//AAAA…(pin de otro servidor / atacante MitM)" https://localhost:8443/
curl: (90) SSL: public key does not match pinned public key
   (exit 90)

$ curl https://localhost:8443/   (sin confiar en el certificado: el cliente lo rechaza)
curl: (60) SSL certificate problem: unable to get local issuer certificate
More details here: https://curl.se/docs/sslcerts.html
```

Simulación de ataque Man-in-the-Middle: se detuvo el servidor real y en el mismo puerto se levantó un servidor impostor (openssl s_server). Su certificado tiene los mismos nombres e IP, pero es de un atacante. Con el servidor real, la app se conecta y el servidor responde; con el impostor, la app aborta la conexión antes de enviar el correo y la contraseña:



Servidor real: conexión HTTPS con pinning (el servidor respondió «correo o contraseña incorrectos»)



Servidor impostor (MitM): la app rechaza su certificado y no envía credenciales

EVIDENCIA log de la app y del servidor impostor (09-android-mitm-logcat.txt)

```
$ adb logcat | grep PRISMA   (servidor impostor con certificado distinto en 192.168.100.31:8443)
11-11 17:39:36.883 15652 15652 I System.out: PRISMA auth network error: SSLHandshakeException
Using default temp DH parameters
ACCEPT
802112F201000000:error:0A000416:SSL routines:ssl3_read_bytes:tls alert certificate unknown:ssl/recor
d/rec_layer_s3.c:923:SSL alert number 46
```

Permisos y configuración del AndroidManifest.xml

- Se eliminaron READ_EXTERNAL_STORAGE y READ_MEDIA_IMAGES: los PDF y las fotos se eligen con el selector del sistema, que no requiere permisos.
- Con tools:node="remove" se evita que una librería agregue permisos peligrosos (cámara, ubicación, contactos, almacenamiento).
- usesCleartextTraffic=false y network_security_config con cleartextTrafficPermitted=false.
- allowBackup=false, fullBackupContent=false y dataExtractionRules: sin respaldos de los datos.
- Solo la actividad de inicio está exportada (es obligatorio para el lanzador).

EVIDENCIA permisos y atributos del APK compilado (08-android-manifiesto.txt)

```
$ aapt2 dump permissions app/androidApp/build/outputs/apk/debug/androidApp-debug.apk
package: com.example.prisma
uses-permission: name='android.permission.INTERNET'
uses-permission: name='android.permission.ACCESS_NETWORK_STATE'
uses-permission: name='android.permission.USE_BIOMETRIC'
```

```
uses-permission: name='android.permission.USE_FINGERPRINT'
permission: com.example.prisma.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION
uses-permission: name='com.example.prisma.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION'

$ aapt2 dump xmltree --file AndroidManifest.xml (atributos de <application>)
        A: http://schemas.android.com/apk/res/android:allowBackup(0x01010280)=false
        A: http://schemas.android.com/apk/res/android:fullBackupContent(0x010104eb)=false
        A: http://schemas.android.com/apk/res/android:usesCleartextTraffic(0x010104ec)=false
        A: http://schemas.android.com/apk/res/android:networkSecurityConfig(0x01010527)=@0x7f110001
        A: http://schemas.android.com/apk/res/android:dataExtractionRules(0x0101063e)=@0x7f110000
```

USE_BIOMETRIC y USE_FINGERPRINT los agrega la librería androidx.biometric y son permisos de nivel normal, no peligrosos. DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION es un permiso de firma que crea AndroidX para proteger sus receptores internos.

Parte 4: Implementación de Web Services

4.1 Arquitectura de servicios: catálogo OpenAPI

El catálogo completo de endpoints está en formato OpenAPI 3.0.3 en server/src/main/resources/openapi/documentation.yaml: 20 rutas, esquemas, códigos de respuesta y esquemas de seguridad. En modo desarrollo, el servidor lo publica con Swagger UI en /swagger; en producción no se expone.

Entidad	GET	POST	PUT	DELETE
Sesión (/auth)	—	register, login, verify-2fa, logout, forgot/reset-password	—	—
Dispositivo biométrico	—	enroll, login, renew, revoke	—	—
Perfil (/user/profile)	Consultar	Crear o actualizar	—	Borrar
Tarjeta (/cards)	Listar	Crear	Editar /{id}	Borrar /{id}
Estado de cuenta (/cards/{id}/documents)	Listar	Subir (multipart)	—	Borrar /{docId}
Ingreso (/incomes)	Listar + total	Crear	Editar /{id}	Borrar /{id}
Resumen (/reports/latest)	Consultar	—	—	—
Nómina IA (/ai/payroll)	—	analyze, upload	—	—

¿Por qué JSON? Es el formato nativo de kotlinx.serialization en el servidor y en las tres apps: las mismas clases de datos del módulo core se usan en ambos lados, con tipos verificados al compilar. Además es ligero, legible y el navegador lo procesa sin conversión. Como siempre se responde application/json con nosniff, el navegador nunca lo interpreta como HTML.

documentation.yaml — fragmento `server/src/main/resources/openapi/documentation.yaml:265`

```
/api/v1/incomes/{id}:
  parameters: [ { $ref: '#/components/parameters/Id' } ]
  put:
    tags: [Ingresos]
    summary: Editar ingreso (si cambia el monto deja de estar auditado)
    requestBody:
      required: true
      content: { application/json: { schema: { $ref: '#/components/schemas/IncomeRequest' } } }
    responses:
      '200': { description: Actualizado, content: { application/json: { schema: { $ref: '#/components/
schemas/IncomeDto' } } } }
      '404': { $ref: '#/components/responses/NotFound' }
  delete:
    tags: [Ingresos]
    summary: Eliminar ingreso
    responses:
      '204': { description: Eliminado }
      '404': { $ref: '#/components/responses/NotFound' }
```

4.2 Integración de APIs de terceros

Google Gemini (auditoría de nómina con IA)

El endpoint /api/v1/ai/payroll/analyze envía los datos de la nómina a la API de Google Gemini (generativelanguage.googleapis.com) para detectar diferencias y dar recomendaciones. La llave de API se lee de GEMINI_API_KEY en server/.env y viaja en la cabecera x-goog-api-key por HTTPS, nunca en la URL, para que no quede en logs ni en el historial. Antes de enviarse, los documentos se anonimizan, y esto requiere el consentimiento obligatorio «EnvioIa» del registro.

GeminiClient.kt — autenticación con token de API `server/src/main/kotlin/com/example/prisma/ai/GeminiClient.kt:63`

```
val request = HttpRequest.newBuilder()
    .uri(
        URI.create(
            "https://generativelanguage.googleapis.com/" +
                    "v1beta/models/$model:generateContent"
        )
    )
    .header("x-goog-api-key", apiKey)
    .header("Content-Type", "application/json")
```

Gmail SMTP (envío de códigos 2FA y de recuperación)

El servidor envía los códigos de verificación por correo con la API SMTP de Gmail (smtp.gmail.com:587). Se conecta con STARTTLS obligatorio y una contraseña de aplicación de Google guardada en server/.env o server/smtp.properties, ambos fuera de git.

EmailService.kt — STARTTLS obligatorio y credenciales externas `server/src/main/kotlin/com/example/prisma/services/EmailService.kt:85`

```
put("mail.smtp.auth", "true")
put("mail.smtp.starttls.enable", "true")
put("mail.smtp.starttls.required", "true")
put("mail.smtp.host", host)
```

EVIDENCIA arranque del servidor con SMTP configurado (correo enmascarado)

```
2026-11-11 17:34:26.362 [main] INFO  io.ktor.server.Application - Correo SMTP configurado: p***@gmai
l.com vía smtp.gmail.com.
```

Archivo de variables de entorno seguro

Los secretos (llave JWT, llave AES, contraseña del keystore TLS, base de datos, Gemini y SMTP) viven solo en server/.env, con permisos 600 e ignorado por git. En el repositorio solo está la plantilla sin valores:

.env.example — plantilla sin secretos `server/.env.example:10`

```
# ---- HTTPS / TLS ----
HTTP_PORT=8080
HTTPS_PORT=8443
KEYSTORE_PATH=certs/prisma-keystore.p12
# Genera una con: openssl rand -base64 24
```

```
KEYSTORE_PASSWORD=

# ---- Autenticación (JWT HS256) ----
# Mínimo 32 caracteres. Genera uno con: openssl rand -base64 48
JWT_SECRET=

# ---- Cifrado de datos personales en reposo (AES-256-GCM) ----
# 32 bytes en Base64. Genera una con: openssl rand -base64 32
# ¡Si la pierdes no podrás leer los datos ya cifrados!
DATA_ENCRYPTION_KEY=

# ---- Base de datos PostgreSQL ----
JDBC_DATABASE_URL=jdbc:postgresql://localhost:5432/prisma_db
DB_USER=postgres
DB_PASSWORD=

# ---- API de terceros: Google Gemini (auditoría de nómina) ----
# https://aistudio.google.com/apikey
GEMINI_API_KEY=
```

.gitignore `.gitignore:21`

```
# Credenciales SMTP locales del servidor (no subir)
smtp.properties

# Secretos del servidor y llave privada TLS (no subir)
.env
!.env.example
server/certs/*.p12
```

CAPTURA PENDIENTE — GEMINI FUNCIONANDO

1. Pega tu llave de https://aistudio.google.com/apikey en server/.env (GEMINI_API_KEY=...) y reinicia el servidor.
2. En la app, abre «Asistente IA», sube un recibo de nómina y captura el resultado del análisis.
3. Captura también el correo con el código 2FA recibido en Gmail (evidencia de la API SMTP).

Inserta aquí la captura.

4.3 Autenticación remota segura: JWT

El inicio de sesión tiene dos pasos: usuario y contraseña, y luego un código 2FA enviado por correo. Al completarlo, el servidor emite un JWT firmado con HMAC-SHA256. El JWT incluye emisor, audiencia, sujeto (id del usuario), un jti aleatorio de 256 bits y una expiración de 12 horas.

En cada petición se verifican la firma, el emisor, la audiencia y la expiración. Además, el jti debe seguir registrado en el servidor, así que cerrar sesión revoca el token aunque su firma siga siendo válida. Las contraseñas nunca viajan en claro: todo el tráfico va por TLS 1.2/1.3 y el puerto HTTP solo redirige.

SessionStore.kt — emisión del JWT `server/src/main/kotlin/com/example/prisma/auth/SessionStore.kt:131`

```
fun createSession(userId: UUID, twoFactorAt: Long = clock()): String {
    purgeExpired()
    val jti = newToken()
    val now = clock()
    val expiresAt = minOf(now + sessionTtlMillis, twoFactorAt + twoFactorWindowMillis)
    sessions[jti] = Session(userId, expiresAt, twoFactorAt, csrfToken = newToken())
    return JWT.create()
        .withIssuer(JWT_ISSUER)
        .withAudience(JWT_AUDIENCE)
        .withSubject(userId.toString())
        .withJWTId(jti)
        .withIssuedAt(Date(now))
        .withExpiresAt(Date(expiresAt))
        .sign(algorithm)
```

EVIDENCIA **JWT real y ataques de manipulación (05-pruebas-ataques-api.txt)**

```
=== 1. JWT (Parte 4.3) ===
encabezado: {"alg":"HS256","typ":"JWT"}
carga útil: {"iss": "prisma-api", "aud": "prisma-clients", "sub": "037dc33d-a6eb-4545-9bc6-0d9e3d075d
4e", "iat": 1794440760, "exp": 1794483960}
[OK ] JWT con 'sub' alterado (sin conocer la llave) se rechaza
        401 {"code":"unauthorized","message":"Sesión inválida o expirada."}
[OK ] JWT con alg=none se rechaza
        401 {"code":"unauthorized","message":"Sesión inválida o expirada."}

=== 9. Cierre de sesión revoca el JWT ===
[OK ] Tras logout, el mismo JWT (firma aún válida) → 401
        logout 204; reuso 401 {"code":"unauthorized","message":"Sesión inválida o expirada."}
```

4.4 Codificación segura y auditoría: SAST y DAST

Pruebas automáticas

La suite del servidor tiene 33 pruebas, todas aprobadas; 12 son nuevas y verifican los controles de esta actividad. Se ejecutan con ./gradlew :server:test.

Prueba (SecurityTest.kt)	Control verificado
httpIsRedirectedToHttps	HTTP responde 301 a https://
securityHeadersArePresent	CSP, X-Frame-Options, nosniff, Referrer-Policy, Server
internalErrorsDoNotLeakStackTraces	Errores 500 sin trazas ni datos internos
accessTokenIsSignedJwtAndTamperingIsRejected	JWT HS256; firma alterada u otra llave → rechazo
logoutRevokesJwtEvenIfSignatureIsValid / expiredJwtIsRejected	Revocación y expiración
cookieAuthenticatedWritesRequireCsrfToken / bearer…	CSRF: 403 sin token o con token falso
textWhitelistRejectsScriptsAndSqlPayloads	Listas blancas contra XSS / SQL
uploadWhitelistChecksRealContentAndSanitizesName	Archivos por contenido real y nombre limpio
personalDataIsEncryptedWithAes256Gcm	AES-256-GCM: IV aleatorio, detección de alteración

Prueba (SecurityTest.kt)	Control verificado
rejectedCorsRequestStillGetsHeadersWithoutError	Regresión del hallazgo de ZAP

EVIDENCIA **resumen del script de ataques contra la API real**

```
Resultado: 26/26 controles verificados
```

SAST — Semgrep

Se analizaron 140 archivos (servidor, app compartida, Android y core) con 74 reglas de los conjuntos Kotlin, Java, secretos, OWASP Top 10 y JWT. No hubo hallazgos en el código. El único aviso es que la actividad principal está exportada en el manifiesto; es un falso positivo aceptado, porque la actividad del lanzador debe ser exportada.

Semgrep 1.157.0 — Análisis estático (SAST) de Prisma

Comando: `semgrep scan --config p/kotlin --config p/java --config p/secrets --config p/owasp-top-ten --config p/jwt server/src app/shared/src app/androidApp/src core/src`

140	74	0	1	0
archivos analizados	reglas ejecutadas	vulnerabilidades en código	hallazgo (manifiesto)	errores

Archivos: 104 .kt, 15 .webp, 9 .xml, 5 .sql, 3 .png, 3 .ttf, 1 .yaml. Conjuntos de reglas: Kotlin, Java, secretos/llaves expuestas, OWASP Top 10 y JWT.

Regla	Ubicación	Severidad	Análisis
exported_activity	app/androidApp/src/main/AndroidManifest.xml:17	WARNING	Falso positivo aceptado: la actividad principal (MAIN/LAUNCHER) debe ser exportada para que Android pueda abrir la app; no recibe datos de otras apps.

Resultado de Semgrep (reporte completo: evidencias/semgrep-resultado.json)

DAST — OWASP ZAP

ZAP 2.17 se ejecutó con su Automation Framework. Importó el catálogo OpenAPI (27 URLs), se autenticó con el JWT de un usuario de prueba y corrió los escaneos pasivo y activo sobre el servidor HTTPS.

Resultado: 0 alertas altas, 0 medias, 1 baja y 3 informativas. La alerta baja es la cookie __Host-prisma_csrf, que no es HttpOnly a propósito: el patrón de doble envío necesita que la app web pueda leerla.

Hallazgo corregido: en la primera ejecución, ZAP provocó un error 500 al enviar una petición CORS con un origen no permitido. El cliente solo recibió el mensaje genérico, pero el log mostró que el plugin de cabeceras intentaba escribir en una respuesta ya enviada. Se corrigió (SecurityHeaders usa onCallRespond), se agregó la prueba de regresión y el segundo escaneo terminó sin errores 500.

Prisma API – OWASP ZAP (DAST)

Sitios: http://localhost:9080 https://localhost:9443

Generado a mié, 11 nov 2026 17:45:26

ZAP Versión: 2.17.0

ZAP by Checkmarx

Sumario de Alertas

Nivel de riesgo	Número de Alertas
Alto	0
Medio	0
Bajo	1
Informativo	3
Falsos Positivos:	0

Insights

Level	Razón	Site	Descripción	Statistic
Bajo	Advertencia		ZAP errors logged - see the zap.log file for details	2
Bajo	Advertencia		ZAP warnings logged - see the zap.log file for details	2
Información	Informativo	http://localhost:9080	Percentage of responses with status code 3xx	100 %
Información	Informativo	http://localhost:9080	Percentage of endpoints with method GET	100 %
Información	Informativo	http://localhost:9080	Count of total endpoints	1
Información	Informativo	https://localhost:9443	Percentage of responses with status code 2xx	7 %
Información	Informativo	https://localhost:9443	Percentage of responses with status code 4xx	92 %
Información	Informativo	https://localhost:9443	Percentage of endpoints with content type application/json	92 %
Información	Informativo	https://localhost:9443	Percentage of endpoints with content type text/plain	3 %
Información	Informativo	https://localhost:9443	Percentage of endpoints with method DELETE	14 %
Información	Informativo	https://localhost:9443	Percentage of endpoints with method GET	21 %
Información	Informativo	https://localhost:9443	Percentage of endpoints with method POST	57 %
Información	Informativo	https://localhost:9443	Percentage of endpoints with method PUT	7 %
Información	Informativo	https://localhost:9443	Count of total endpoints	28
Información	Informativo	https://localhost:9443	Percentage of slow responses	4 %

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alertas

Nombre	Nivel de riesgo	Número de Instancias
Cookie Sin Flag HttpOnly	Bajo	1
Petición de Autenticación Identificada	Informativo	2
Reexaminar las Directivas de Control de Caché	Informativo	1
Tecnología Detectada - HSTS	Informativo	1

Detalles de la Alerta

Reporte de OWASP ZAP (completo en evidencias/zap-dast-reporte.html)

Resumen de cumplimiento

Criterio	Indicador	Evidencia
1. Protección de datos	EncryptedSharedPreferences + MODE_PRIVATE en móvil	1.1
	Hash BCrypt con sal	1.2
	Cookies httpOnly y secure	1.3
	Errores sin trazas / debug = false	1.4
2. OWASP Top 10	Consultas 100% parametrizadas	2.2 + Semgrep
	Escape de salidas contra XSS	2.1
	Listas blancas en backend	2.1
	Tokens anti-CSRF	2.4
3. SSL/TLS	Redirección HTTP → HTTPS	3.1
	Solo TLS 1.2 / 1.3	3.1
	HSTS, CSP, X-Frame-Options, nosniff	3.2
4. Seguridad móvil	Validación de certificado en la app	3.3
	Certificate Pinning contra MitM	3.3 (prueba con impostor)
	Permisos peligrosos restringidos en el manifiesto	3.3
5. Web Services	APIs de terceros (Gemini, Gmail SMTP)	4.2
	Servicios cliente/servidor + .env seguro	4.2, 3.3
	Config. de base de datos y cookies + capturas	1.2, 1.3
	Controladores + pruebas de inyección fallidas	2.2, 2.3
	Inspección de red del navegador + conexión móvil	3.2, 3.3

Conclusión

[Borrador para que el equipo lo ajuste con su experiencia] Lo más complejo fue llevar HTTPS a las tres plataformas a la vez. Activar TLS en el servidor fue sencillo; lo difícil fue que la app de Android, la de iOS y la web confiaran en un certificado autofirmado sin desactivar la validación, que es justo lo que se busca evitar. La solución fue anclar el certificado (Certificate Pinning) y calcular el pin al compilar. La prueba con un servidor impostor mostró que la app rechaza el ataque Man-in-the-Middle antes de enviar cualquier credencial.

Lo más determinante fue probar los controles con ataques reales, no solo escribirlos. Las pruebas de IDOR, CSRF e inyección confirmaron que cada endpoint filtra por el usuario del JWT, y el escaneo de OWASP ZAP encontró un error 500 que habíamos introducido nosotros. Esto nos enseñó que la seguridad se verifica de forma continua: con pruebas automáticas que se ejecutan en cada cambio y con herramientas SAST y DAST, y no solo con revisar el código.